Provider callback behavior, identity-linking assumptions, signup consistency, mail delivery notifications and external-service responses have not been runtime-tested. Human verification remains pending.

Conclusion: Of these three flows, identity-linking coordination and the explicit Action Mailer observer registration are the only genuinely new pattern candidates supported by the inspected code. Both remain unconfirmed until their defining responsibility or notification mechanism is verified.

Before moving to the frontend source area, verify the two candidates from the external-integration investigation.
The candidates are:

1. GRASP Controller — OAuth Sign-in and Identity Linking Coordination
2. Observer — Outbound Mail Delivery Logging

Evaluate them independently.
For the OAuth GRASP Controller candidate:

1. Identify the system event handled by `SessionsController#create`.
2. Separate what OmniAuth has already completed before control reaches the application from what DaPlanStan decides afterward.
3. Trace the application decisions around provider identity lookup, email-based user lookup, `UserIdentity` creation, session establishment, pending traveler acceptance and account provisioning.
4. Identify the actual participants and give reliable source paths and line ranges.
5. Explain why this is more than a conventional OAuth callback.
6. Give the strongest argument that it is simply procedural post-authentication account handling rather than a meaningful GRASP Controller responsibility.
7. Compare it with the already confirmed GRASP Controller instances and decide whether it represents an independent system-event collaboration.
8. Give a final decision of CONFIRMED, REJECTED or UNCERTAIN.

For the Outbound Mail Delivery Logging Observer candidate:

1. State the essential Observer mechanism that must exist.
2. Trace the relationship created by `ActionMailer::Base.register_observer(MailLogger)`.
3. Identify the subject or publisher, observer, registration operation and notification callback using actual source symbols.
4. Determine whether `MailLogger.delivered_email(message)` is called because of a real Action Mailer observer contract rather than merely by naming convention.
5. Separate the part implemented by DaPlanStan from the observer infrastructure implemented by Rails.
6. Explain whether framework-mediated Observer implementations are still defensible pattern instances when the application supplies the concrete observer and explicitly registers it.
7. Compare this with the unresolved Turbo-mediated Observer candidate. Do not merge them unless they are actually the same collaboration.
8. Give the strongest argument against counting it.
9. Give a final decision of CONFIRMED, REJECTED or UNCERTAIN.

Do not require runtime testing just to establish a static structural pattern if the source and framework contract are sufficient.
However, keep runtime verification status separate.
At the end provide:

1. One decision table for both candidates.
2. Participant-role mapping for any confirmed instance.
3. Stable instance names.
4. Source evidence to preserve.
5. Counting decision.
6. Any remaining uncertainty that belongs to runtime verification rather than pattern identification.

Do not investigate any new pattern types in this prompt.

Worked for 58s

DaPlanStan — External Integration Pattern Verification

Candidate Review 05 · Original source archive · 23CSE455

1. Verification decisions